🔍 Observability

Logging, Metrics & Tracing — Module 9 of 13

Enterprise EKS Platform

📋 Agenda

📚 Concepts

- Three Pillars of Observability
- Container Logging Model
- Kubernetes Metrics Architecture
- Distributed Tracing Fundamentals
- SLOs and Error Budgets

🛠️ Hands-On

- kubectl logs and centralized logging
- Prometheus & Grafana stack
- OpenTelemetry instrumentation
- Alertmanager configuration
- CloudWatch Container Insights

Three Pillars of Observability

Logs, Metrics, and Traces

🤔 Observability vs. Monitoring

Monitoring

- Predefined dashboards and alerts
- Answers **known** questions
- "Is CPU above 80%?"
- Reactive: tells you *what* is broken

Observability

- Explore system state dynamically
- Answers **unknown** questions
- "Why are requests from region X slow?"
- Proactive: tells you *why* it's broken

Goal: Build systems where internal state is externally queryable without deploying new code.

📈 The Three Pillars

📜 Logs

- What happened
- Immutable records
- High cardinality

📊 Metrics

- How much / how fast
- Aggregatable
- Low storage cost

🔗 Traces

- Where time is spent
- Causal relationships
- Cross-service visibility

🔄 Pillars Working Together

Typical Debugging Flow

1. **Metrics** alert fires — error rate exceeds threshold
2. **Traces** reveal which service in the call chain is slow
3. **Logs** from that service show the root cause (e.g., DB timeout)

> **Correlation is key:** Use trace IDs to link logs, metrics, and traces for the same request.

📜 Logging in Kubernetes

Container stdout/stderr to centralized stores

📦 Container Logging Model

How Kubernetes Captures Logs

- Containers write to `stdout` and `stderr`
- Container runtime redirects streams to a logging driver
- Kubelet stores logs at `/var/log/containers/`
- Log rotation managed by kubelet (`containerLogMaxSize`, `containerLogMaxFiles`)
- Logs are **ephemeral** — lost when pod is deleted

> **Key implication:** Never rely on local container logs. Ship them to a durable store.

🏗️ Centralized Logging Architecture

DaemonSet-based Collection Pattern

Node agent reads log files → enriches with K8s metadata → ships to backend

📦 Collectors

- **Fluent Bit** — lightweight, C-based
- **Fluentd** — flexible, plugin-rich
- **Vector** — high-performance, Rust-based
- **CloudWatch Agent** — AWS-native

🗃️ Backends

- **Elasticsearch + Kibana** (ELK/EFK)
- **Grafana Loki** — like Prometheus for logs
- **AWS CloudWatch Logs**
- **Datadog / Splunk**

📝 Structured Logging Best Practices

❌ Unstructured

- Free-text log lines
- Requires fragile regex to parse
- Hard to filter, aggregate, or alert on

✅ Structured (JSON)

- Consistent key/value fields
- timestamp, level, message, service
- Indexable and queryable without regex

Why structured? Enables filtering, aggregation, and correlation. Always include `trace_id` for cross-pillar linkage.

☁️ CloudWatch Logs with EKS

EKS Log Groups Structure

- `/aws/eks/cluster-name/cluster` — control plane logs
- `/aws/containerinsights/cluster-name/application` — pod logs
- `/aws/containerinsights/cluster-name/host` — node-level logs
- `/aws/containerinsights/cluster-name/dataplane` — kubelet, kube-proxy

CloudWatch Logs Insights provides a SQL-like query language to filter, sort, and aggregate log fields (including injected Kubernetes metadata) across log groups.

🔬 Logging to Splunk on EKS

Fluent Bit → Splunk HEC Pipeline

Fluent Bit ships logs to Splunk via the `splunk` output plugin and HTTP Event Collector (HEC)

Key detail: HEC token is stored in a Kubernetes Secret and injected via `envFrom`. Index-per-namespace routing uses Fluent Bit's `Rewrite_Tag` filter to direct logs to the correct Splunk index.

Metrics in Kubernetes

Measuring system health and performance

🏗️ Kubernetes Metrics Architecture

📏 Resource Metrics

- CPU & memory per pod/node
- Powers HPA & VPA
- Lightweight, in-memory

🎯 Custom Metrics

- App-specific (queue depth, etc.)
- Prometheus Adapter
- Enables custom HPA scaling

🌎 External Metrics

- Outside-cluster sources
- SQS queue length, etc.
- CloudWatch metrics adapter

🔥 Prometheus Fundamentals

Pull-Based Architecture

- Prometheus **scrapes** /metrics endpoints on a schedule
- Service discovery via Kubernetes API (pods, services, endpoints)
- Time-series database with PromQL query language
- Built-in alerting via Alertmanager integration

Four metric types: Counter (only goes up), Gauge (arbitrary values), Histogram (bucketed distributions), Summary (quantile calculations)

🔭 ServiceMonitor Configuration

Declarative Scrape Targets

- **ServiceMonitor** is a CRD from the Prometheus Operator
- A label **selector** matches the target Kubernetes Service
- Defines the metrics port, path, and scrape interval
- Operator auto-updates Prometheus config — no manual edits

> **Declarative over imperative:** add scrape targets by creating resources, not by editing a central Prometheus config file.

📊 Grafana Dashboards

🎭 Essential Dashboards

- **Cluster Overview** — node health, capacity
- **Namespace Resources** — CPU, memory, network
- **Pod Details** — restarts, OOMKills, throttling
- **Persistent Volumes** — usage, IOPS
- **Networking** — DNS, ingress, service mesh

📋 USE & RED Methods

USE (Infrastructure):
Utilization, Saturation, Errors

RED (Services):
Rate, Errors, Duration

🔧 kube-state-metrics & node-exporter

⚙️ kube-state-metrics

Generates metrics from Kubernetes API objects

- Deployment replicas desired vs. available
- Pod phase, conditions, restarts
- Job/CronJob success/failure counts
- Resource requests and limits

💻 node-exporter

Exports hardware and OS metrics from nodes

- CPU, memory, disk, network I/O
- Filesystem usage and inodes
- System load averages
- Runs as DaemonSet on each node

> **Together:** kube-state-metrics tells you what Kubernetes *thinks*; node-exporter tells you what the node *experiences*.

☁️ CloudWatch Container Insights

AWS-Native Observability for EKS

- Pre-built dashboards for clusters, nodes, pods, services
- Automatic collection of CPU, memory, disk, network metrics
- Deployed via **CloudWatch Agent** or **ADOT Collector** (DaemonSet)
- Integrated with CloudWatch Alarms and SNS notifications
- Enhanced observability mode adds Prometheus-compatible metrics

🔗 Distributed Tracing

Following requests across service boundaries

🔭 Tracing Concepts

Key Terminology

- **Trace** — end-to-end journey of a request
- **Span** — a single operation within a trace
- **Parent/Child** — spans form a tree structure
- **Context Propagation** — passing trace IDs across service boundaries

Anatomy: A trace is a tree of timed spans — e.g. API Gateway → Auth Svc, then Order Svc → DB Query and Cache — revealing where time is spent at each hop.

🌎 OpenTelemetry (OTel)

📚 SDKs

- Auto-instrumentation
- Manual span creation
- Java, Python, Go, Node.js

📦 Collector

- Receives, processes, exports
- Pipeline: receivers → processors → exporters
- Deploy as sidecar or DaemonSet

🔌 OTLP

- Standard wire protocol
- gRPC and HTTP transport
- Covers logs, metrics, traces

🔭 Tracing Backends

🔎 Jaeger

- CNCF graduated project
- Full trace search and comparison
- Dependency graph visualization
- Supports Elasticsearch or Cassandra storage
- Self-hosted, full control

☁️ AWS X-Ray

- Managed service, no infrastructure
- Native AWS service integration
- Service map auto-generated
- Trace groups and filter expressions
- Integrates via ADOT Collector

Recommended pattern: Use X-Ray on EKS for AWS-native visibility. Add Jaeger for advanced trace analysis or self-hosted retention.

🚨 Alerting & Incident Response

From detection to resolution

Enterprise Alerting Destinations

Netcool (Webhook)

- Enterprise event management
- Alertmanager `webhook_configs`
- POST JSON to Netcool probe REST endpoint
- Maps severity → Netcool class

Email (SMTP)

- Built-in `email_configs` receiver
- Corporate SMTP relay
- HTML templates for context
- Best for low-urgency / ticket alerts

WebEx (Webhook)

- WebEx Teams Incoming Webhook
- Uses `webhook_configs`
- Adaptive Card payload
- Team Spaces for on-call visibility

Routing tree strategy: `severity: critical` → Netcool + WebEx | `severity: warning` → Email + WebEx | `severity: info` → Email only

🎯 SLOs and Error Budgets

Definitions

- **SLI** — Service Level Indicator (measured metric)
- **SLO** — Service Level Objective (target value)
- **SLA** — Service Level Agreement (contractual)
- **Error Budget** — tolerable failure amount

📈 Example

Component	Value
SLI	% requests < 200ms
SLO	99.9% over 30 days
Error Budget	43.2 min/month

> Budget remaining: 28.7 min

🔥 Burn Rate Alerting

Multi-Window Burn Rate Strategy

Alert based on how fast you're consuming your error budget, not just on raw error rate

Severity	Burn Rate	Long Window	Short Window	Budget Consumed
Page	14.4x	1 hour	5 min	2% in 1 hour
Page	6x	6 hours	30 min	5% in 6 hours
Ticket	3x	1 day	2 hours	10% in 1 day
Ticket	1x	3 days	6 hours	10% in 3 days

🛠️ Systematic Debugging Workflow

Step-by-Step Approach

1. **Alert fires** — check severity, read runbook
2. **Metrics** — identify affected service, scope of impact
3. **Traces** — find slow/failing spans, isolate the service
4. **Logs** — search by trace ID for exact error details
5. **Kubernetes state** — `kubectl get events`, pod status, resource pressure
6. **Mitigate** — rollback, scale, or isolate, then fix root cause

> **Anti-pattern:** Jumping straight to `kubectl logs` without first narrowing the scope via metrics and traces.

📚 Module 9 Summary

📜 Data Collection

- Logs: stdout/stderr, Fluent Bit, CloudWatch
- Metrics: metrics-server, Prometheus, Container Insights
- Traces: OpenTelemetry, X-Ray, Jaeger
- Structured logging with trace IDs for correlation

🚨 Operational Response

- Alertmanager routes alerts by severity
- SLOs define measurable reliability targets
- Burn rate alerts catch budget consumption trends
- Systematic debugging: metrics → traces → logs

👨‍💻 Lab 9: Observability

Hands-On Exercise

- Explore kubectl logs flags and multi-container pod logging
- Deploy structured JSON logging and filter with jq
- Run PromQL queries for CPU, memory, and pod metrics
- Navigate Grafana dashboards to investigate workload resources
- Create a PrometheusRule alert for pod restarts
- Debug a failing application using events, logs, and metrics

> **Duration:** ~45-55 minutes

🎯 Key Takeaways

1. Correlate across all three pillars. Use trace IDs to link logs, metrics, and traces. Without correlation, you have three separate silos.

2. Structured logging is essential. JSON logs with consistent fields enable automated parsing, filtering, and alerting.

3. Alert on SLOs, not symptoms. Burn rate alerting reduces noise and focuses on user-facing impact.

4. Instrument with OpenTelemetry. Vendor-neutral instrumentation decouples your code from any specific backend.

? Questions & Discussion

Module 9: Observability — Logging, Metrics & Tracing

Up Next: Module 10 — Health Probes & Application Debugging